



PYTHON BOOTCAMP

www.jomhack.com

OBJECT ORIENTED PROGRAMMING



Inheritance:

- Relationship between classes
- Child classes can override or extend parent functionality

Polymorphism:

- Using the same interface for different types
- Same method can behave differently for different objects

OBJECT ORIENTED PROGRAMMING



Inheritance:

- Relationship between classes
- Child classes can override or extend parent functionality

```
77 # Inheritance
78 class Shape: # Parent class
79     def __init__(self, name):
80         self.name = name
81
82     def area(self):
83         return 0
84
85 class Circle(Shape): # Child inherits from Shape
86     def __init__(self, radius):
87         super().__init__("Circle")
88         self.radius = radius
89
90     def area(self): # Override parent method
91         return 3.14 * self.radius * self.radius
92
93 class Square(Shape): # Child inherits from Shape
94     def __init__(self, side):
95         super().__init__("Square")
96         self.side = side
97
98     def area(self): # Override parent method
99         return self.side * self.side
100
101 # Both Circle and Square inherit 'name' attribute from Shape
102 circle = Circle(5)
103 square = Square(4)
104
105 print(circle.name) # "Circle" (inherited from Shape)
106 print(square.name) # "Square" (inherited from Shape)
```

OBJECT ORIENTED PROGRAMMING



Polymorphism:

- Using the same interface for different types
- Same method can behave differently for different objects

```
108 # Polymorphism
109 def print_area(shape): # Takes any Shape
110     print(f"{shape.name} area: {shape.area()}")
111
112 # Same method call, different behaviors
113 print_area(circle) # "Circle area: 78.5"
114 print_area(square) # "Square area: 16"
115
116 # Or with a list
117 shapes = [Circle(3), Square(5), Circle(2)]
118 for shape in shapes:
119     print_area(shape) # Same code, different results
```